

TIMS Backend Architecture & 5-Member Task Division

Backend-only implementation plan using Express.js, PostgreSQL and raw SQL

1. Backend Technology Stack

- Backend: Node.js + Express.js
- Database: PostgreSQL
- Database access: pg (node-postgres) using raw SQL
- Authentication: JWT
- Password hashing: bcrypt
- File uploads: Multer
- Environment variables: dotenv
- API style: REST API
- Prisma is NOT used.

2. Backend Architecture

React Frontend → Express.js Backend → pg / Raw SQL → PostgreSQL

The Express backend receives API requests, authenticates and authorizes users, runs business logic, executes parameterized SQL through the PostgreSQL connection pool, and returns API responses.

3. Backend File Structure

```
backend/
├── src/
│   ├── config/
│   │   ├── db.js
│   │   └── env.js
│   ├── middleware/
│   │   ├── auth.js
│   │   ├── upload.js
│   │   └── error.js
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── complaintController.js
│   │   ├── clerkController.js
│   │   ├── workOrderController.js
│   │   ├── contractorController.js
│   │   ├── deptHeadController.js
│   │   └── approvalController.js
```

```
|
|
|   ├── financeController.js
|   ├── invoiceController.js
|   └── paymentController.js
|
|   └── services/
|       ├── slaService.js
|       ├── duplicateService.js
|       ├── estimateService.js
|       ├── approvalService.js
|       ├── invoiceService.js
|       ├── verificationService.js
|       └── notificationService.js
|
|   └── routes/
|       ├── authRoutes.js
|       ├── complaintRoutes.js
|       ├── clerkRoutes.js
|       ├── workOrderRoutes.js
|       ├── contractorRoutes.js
|       ├── deptHeadRoutes.js
|       └── financeRoutes.js
|
|   └── utils/
|       ├── generateId.js
|       └── calculateAmount.js
|
|   └── server.js
|
|   └── uploads/
|
|   └── .env
|
|   └── package.json
|
|   └── README.md
```

4. Common Backend Setup — Do Together

- Express.js project setup
- PostgreSQL pg connection pool
- Environment variables
- JWT authentication
- bcrypt password hashing
- Authentication middleware
- Role-based access control
- Township-level authorization
- Error-handling middleware
- Multer/file-upload middleware
- Common API response and error format
- Git branches and integration

5. Authentication & Authorization

Login flow:

1. Email + password
2. Verify password using bcrypt
3. Generate JWT
4. Return token and basic user information
5. Use JWT on protected API requests

The authenticated request should make information such as user ID, role, township ID, department ID and contractor ID available to the backend. RBAC should restrict each API according to the user's role.

6. Multi-Township Authorization

Every protected backend operation must verify that the resource belongs to the authenticated user's township. Frontend hiding alone is not sufficient.

Core rule: req.user.townshipId must match the township of the requested resource.

7. Member 1 — RWA Backend

Responsibility: Complaint creation + complaint tracking + verification/dispute

Main files:

- controllers/complaintController.js
- routes/complaintRoutes.js
- services/verificationService.js

APIs:

- POST /api/v1/complaints
- GET /api/v1/complaints
- GET /api/v1/complaints/:id
- POST /api/v1/complaints/:id/verify
- POST /api/v1/complaints/:id/dispute

Backend logic:

- Create complaint
- Upload complaint photos
- Calculate/set SLA
- Get user's complaints
- Get complaint details
- Track complaint status
- Confirm completed work

- Dispute completed work
- Reopen complaint
- Store dispute reason
- Handle verification status

8. Member 2 — Desk Clerk Backend

Responsibility: Complaint triage → validation → work order → contractor assignment → SLA

Main files:

- controllers/clerkController.js
- controllers/workOrderController.js
- routes/clerkRoutes.js
- routes/workOrderRoutes.js
- services/duplicateService.js
- services/slaService.js

APIs:

- GET /api/v1/clerk/triage-queue
- GET /api/v1/clerk/duplicate-check
- POST /api/v1/clerk/validate/:id
- POST /api/v1/work-orders
- POST /api/v1/clerk/escalate/:id

Backend logic:

- Get complaints awaiting validation
- Validate complaint
- Reject complaint
- Reroute complaint
- Check nearby duplicate complaints
- Link duplicate to master complaint
- Create work order
- Assign contractor
- Set work-order priority
- Monitor SLA
- Detect SLA breach
- Escalate delayed work

The duplicate-check rule defined for TIMS is a 100 m radius check.

9. Member 3 — Field Contractor Backend

Responsibility: Inspection → AMC estimate → execution → completion

Main files:

- controllers/contractorController.js
- routes/contractorRoutes.js
- services/estimateService.js

APIs:

- GET /api/v1/contractor/jobs
- POST /api/v1/contractor/estimates
- PATCH /api/v1/contractor/jobs/:id/status
- POST /api/v1/contractor/evidence

Backend logic:

- Get contractor's assigned jobs
- View work-order details
- Submit site inspection
- Upload before-repair evidence
- Retrieve applicable AMC rates
- Create estimate
- Calculate estimate amount
- Submit estimate for approval
- Update work status
- Record actual work performed
- Record actual labour/service quantities
- Upload after-repair evidence
- Submit completion
- Handle reopened/rework jobs

Important: contractor rates must come from the applicable AMC rate card. The backend should calculate quantity × official rate instead of trusting a rate supplied by the frontend.

10. Member 4 — Department Head Backend

Responsibility: Employee management → approvals → escalations → department analytics

Main files:

- controllers/deptHeadController.js
- controllers/approvalController.js
- routes/deptHeadRoutes.js
- services/approvalService.js

APIs:

- GET /api/v1/dept-head/approvals
- POST /api/v1/dept-head/approvals/:id

- GET /api/v1/dept-head/staff
- POST /api/v1/dept-head/staff
- GET /api/v1/dept-head/analytics

Backend logic:

- Get department employees
- Create Desk Clerk account
- Activate/suspend employees
- Get pending estimates
- Approve estimates
- Request estimate revision
- Handle approval rules
- Handle SLA escalations
- View department performance
- View department expenditure
- View contractor performance

When a Department Head creates an employee, the backend should automatically assign role = DESK_CLERK, the Department Head's department, and the Department Head's township. Arbitrary role selection should not be allowed.

11. Member 5 — Finance & AMC Backend

Responsibility: Contractors → AMC → rate cards → invoice audit → payments

Main files:

- controllers/financeController.js
- controllers/invoiceController.js
- controllers/paymentController.js
- routes/financeRoutes.js
- services/invoiceService.js

APIs:

- GET /api/v1/finance/invoices
- GET /api/v1/finance/invoices/:id/3-way-check
- POST /api/v1/finance/invoices/:id/authorize
- POST /api/v1/finance/invoices/:id/flag-variance
- GET /api/v1/finance/rate-cards

Backend logic:

- Register contractor
- Activate/deactivate contractor
- Manage contractor details

- Associate contractor with AMC
- Create/update/renew/expire AMC
- Check contract validity
- Add/update/deactivate rates
- Retrieve applicable rate
- Check effective dates
- Retrieve invoice
- Compare approved estimate with invoice
- Compare actual work with invoice
- Check AMC rate
- Calculate variance
- Flag incorrect invoice
- Approve valid invoice
- Authorize payment
- Record payment
- View payment status/history

12. Backend Business Rules

- Use parameterized SQL queries (\$1, \$2, etc.); never concatenate user input into SQL.
- Use database transactions for operations that update multiple related records.
- Keep complex business logic in services instead of putting everything in controllers.
- Every protected API must enforce township-level authorization.
- Contractor estimates must use official AMC rates.
- The backend should calculate estimate amounts from quantity and the official rate.
- Store a rate snapshot with estimate items so historical estimates are not changed when AMC rates change.
- Approval thresholds should be configurable rather than hard-coded.
- Inventory/stock management is not part of the backend scope; record actual labour/service quantities instead.
- GPS is deferred and is not required in the current backend.
- PaymentBatch is not required for the basic Phase 1 payment workflow.
- Use consistent status and role values across all modules.
- Important actions should be recorded in audit logs.

13. Common Services

Service	Purpose
slaService.js	Calculate SLA deadlines and determine on-time, at-risk and breached states.
duplicateService.js	Find possible duplicate complaints and link duplicates to a master complaint.
estimateService.js	Retrieve AMC rates and calculate estimate amounts.
approvalService.js	Apply approval rules and process

	approvals/revisions.
invoiceService.js	Perform invoice checks, rate matching and variance calculation.
verificationService.js	Process RWA verification, dispute and reopening.
notificationService.js	Shared service for notifications when notification functionality is added.

14. Recommended Development Order

- Set up Express + PostgreSQL + pg.
- Set up authentication, JWT, RBAC and township authorization.
- Set up common middleware, file upload and error handling.
- Finalize shared API conventions and status/role values.
- M1–M5 implement their backend modules in parallel.
- Integrate the complete complaint-to-payment flow.
- Test APIs using Postman or Thunder Client.
- Fix cross-module authorization and transaction issues.
- Perform end-to-end integration testing.

15. Final 5-Member Backend Ownership

Member	Backend Module	Main Responsibility
M1	RWA Backend	Complaints + Verification
M2	Desk Clerk Backend	Triage + Work Orders + SLA
M3	Contractor Backend	Inspection + Estimates + Execution
M4	Department Head Backend	Employees + Approvals + Escalation
M5	Finance & AMC Backend	Contractors + AMC + Rates + Invoices + Payments

16. Final Backend Ownership Summary

- M1 → RWA Backend
- M2 → Desk Clerk Backend
- M3 → Contractor Backend
- M4 → Department Head Backend
- M5 → Finance & AMC Backend